

High-Throughput and Many-Task Computing: Slurm Edition

COMPLECS Team

<https://bit.ly/COMPLECS>

<https://github.com/sdsc-complecs>

SDSC
SAN DIEGO SUPERCOMPUTER CENTER

UC San Diego

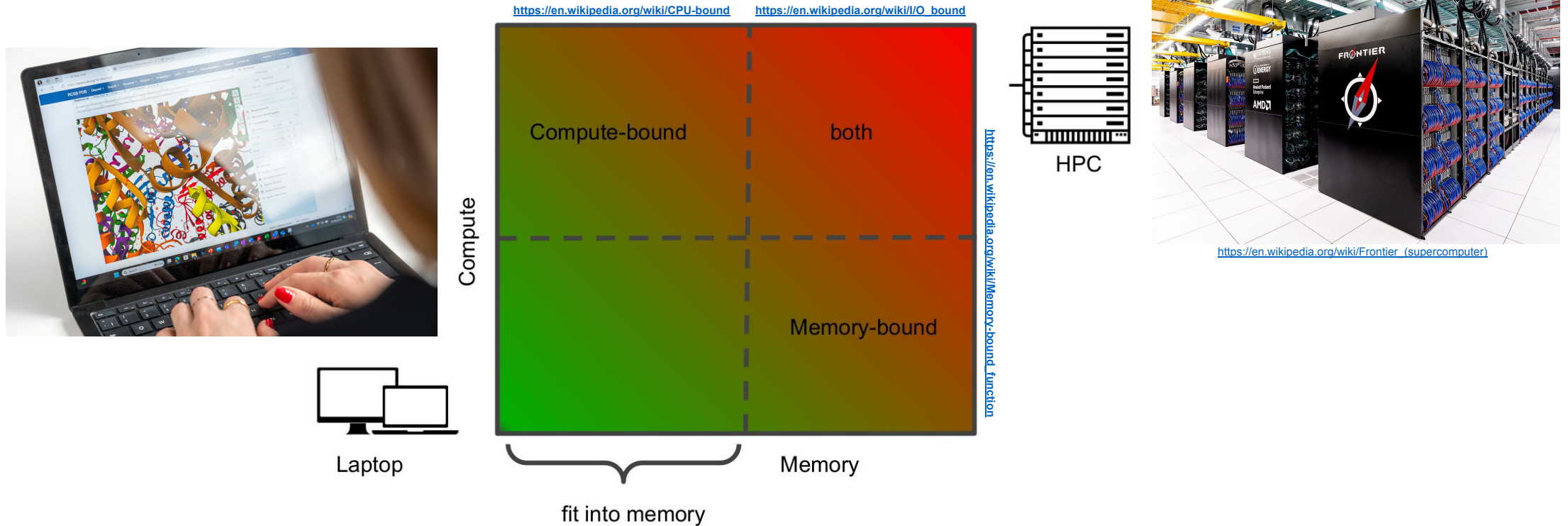
About COMPLECS

COMPLECS (COMPrehensive Learning for end-users to Effectively utilize CyberinfraStructure) is a new SDSC program where training will cover the ***non-programming*** skills needed to effectively use supercomputers. Topics include parallel computing concepts, Linux tools and bash scripting, security, batch and interactive computing, how to get help, and data management.

COMPLECS is supported by NSF [CISE/OAC-2320934](https://www.nsf.gov/awardsearch/showAward.do?awardNumber=CISE/OAC-2320934)



Why COMPLECS?



Your computational workloads can no longer be run effectively on your personal computing resources such as your laptop or desktop workstation, and this is your first time moving your research to a high-performance computing system.

COMPLECS: Batch Computing Series

Working with the Linux Scheduler

How to interact with the Linux scheduler and run your research workloads on your personal computer, a shared workstation, or even a high-performance computing system.

Getting Started with Batch Job Scheduling: Slurm Edition

How to schedule your batch jobs on high-performance computing systems using the Slurm Workload Manager.

High-Throughput and Many-Task Computing Workflows: Slurm Edition

How to build and run your high-throughput and many-task computing workflows on high-performance computing systems using the Slurm Workload Manager.





High-Throughput and Many-Task Computing: Slurm Edition

Learning Goals:

- What is a high-throughput computing?
- What is many-task computing?
- How are they different from traditional high-performance computing workloads?
- How to use Slurm job arrays
- How to use Slurm job dependencies

Prerequisite Knowledge:

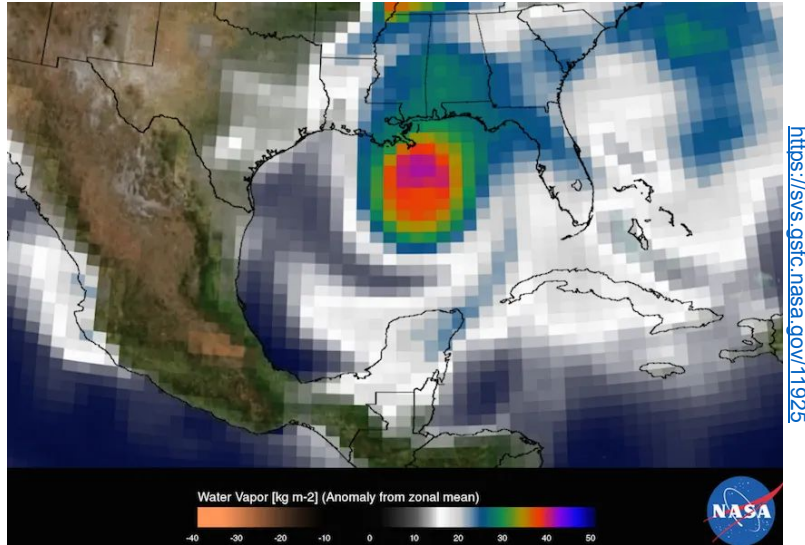
- [Parallel Computing Concepts](#)
- [HPC Hardware](#)
- [Intermediate Linux and Shell Scripting](#)
- [Getting Started with Batch Job Scheduling](#)

Table of Contents

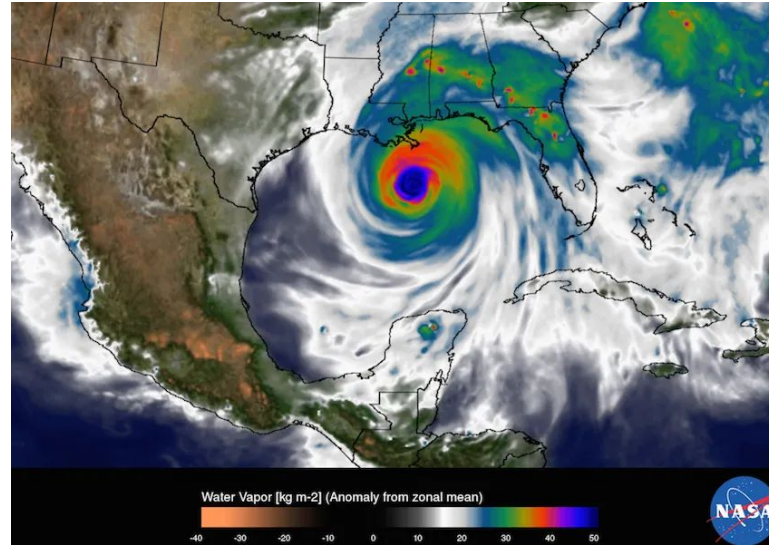
- **High-Throughput and Many-Task Computing**
- **Batch Job Scheduling with Slurm**
- **Job Arrays**
- **Job Dependencies**
- **Job Bundling**
- **Distributed High-Throughput Computing Resources**
- **Many-Task Computing and Workflow Managers**
- **Questions & Answers (30 min)**

High-Performance Computing (HPC)

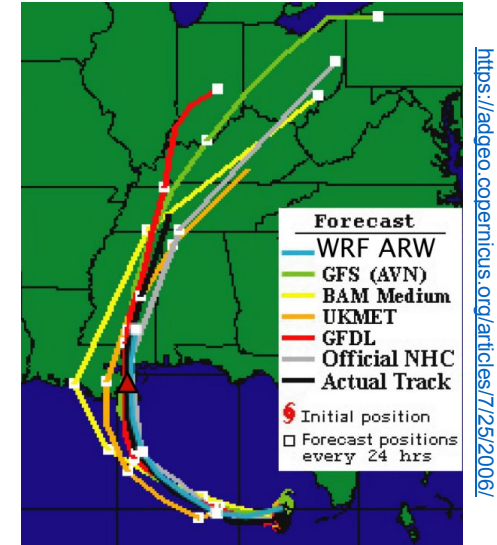
High-performance computing (HPC) uses supercomputers and computer clusters to solve advanced computation problems.



Hurricane Katrina @ 50-km (2005) resolution



Hurricane Katrina @ 6.25-km (2015) resolution



4-km WRF initialized 27 Aug 2005 00 UTC

Key advantages of leveraging HPC for your research:

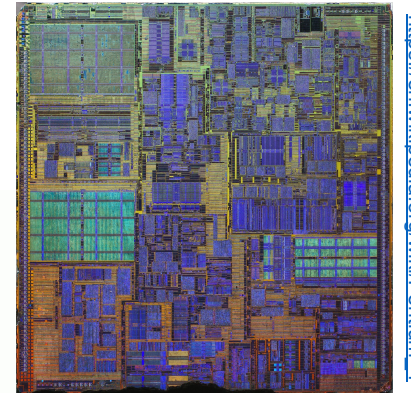
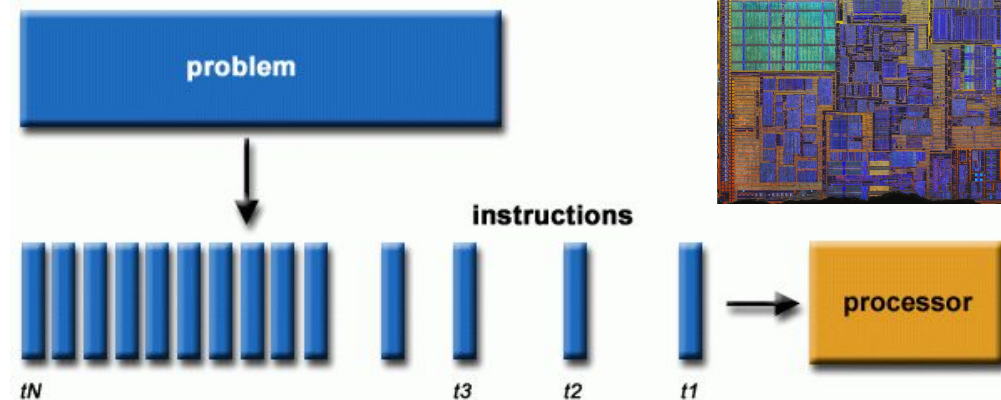
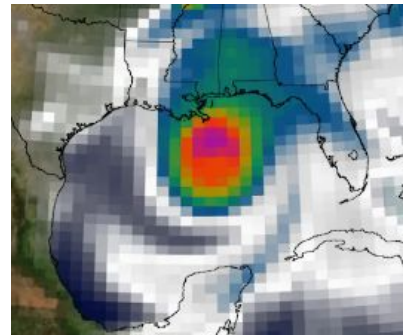
- **Speed** - Solve a problem more quickly
- **Scale** - Solve a larger, more complex problem
- **Throughput** - Solve many (simple) problems more quickly

Serial vs. Parallel Computation

In general, most software applications implement a set of [sequential algorithms](#), where each problem can be solved by a discrete series of instructions that operate on data and execute one after another on a [single processor \(core\)](#). This is known as **serial computing**.

The time required to complete a given problem grows as the complexity of the algorithms used and the total amount of data to be processed increases.

How can you speed things up?



https://en.wikipedia.org/wiki/Pentium_4

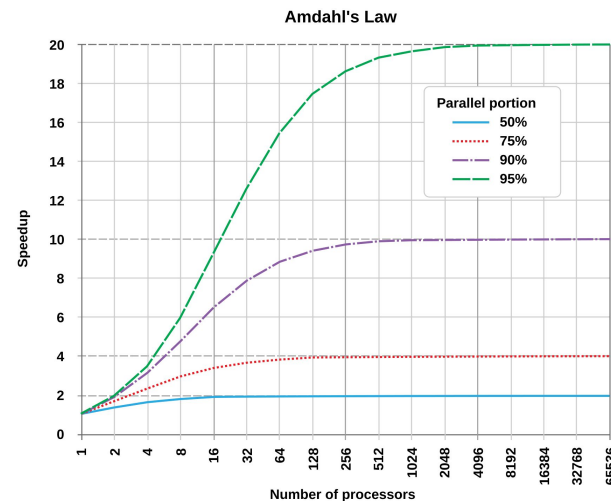
<https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial>

Serial vs. Parallel Computation

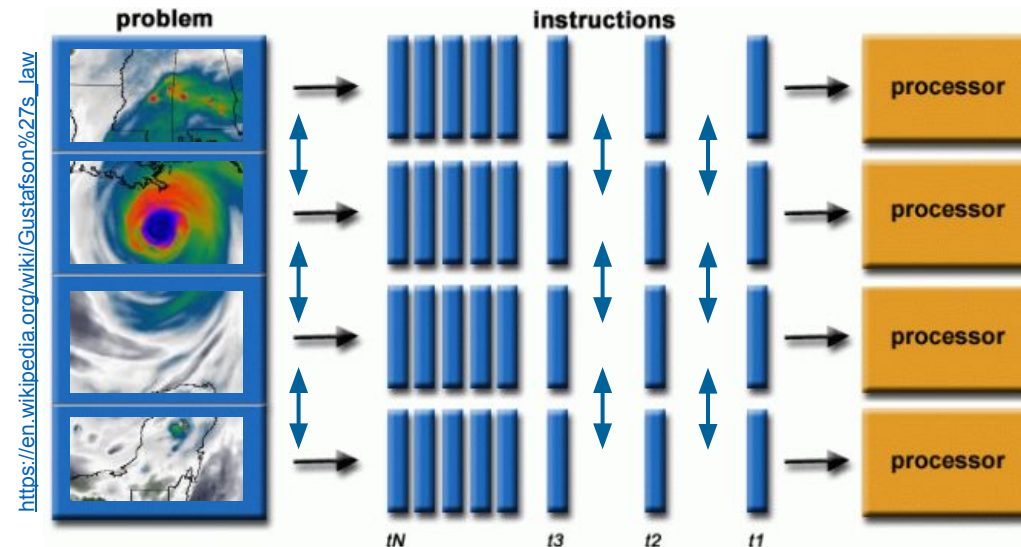
You can take advantage of parallel computing when a problem can be decomposed into a set of discrete subtasks that can be solved concurrently, where each subtask consists of a series of instructions that operate on data *independently* and may be executed simultaneously on different processor cores.

Almost every computer you encounter today, including your personal laptop, desktop workstation, and smartphone, run on multi-core processors capable of parallel computing.

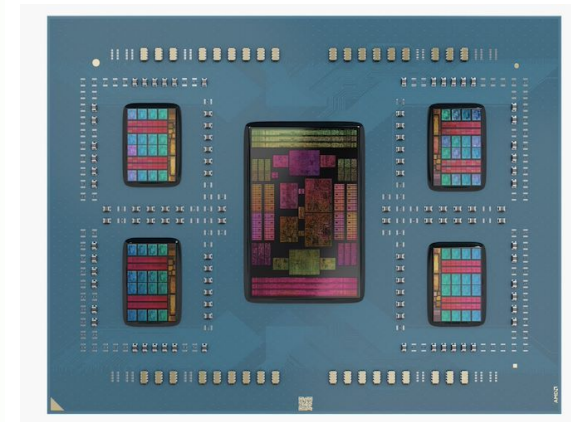
So, how fast can we go?



https://en.wikipedia.org/wiki/Amdahl%27s_law



<https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial>



High-Throughput Computing (HTC)

High-throughput computing (HTC) aims to solve a problem over time by completing *many similar*, usually *small, independent* subtasks.

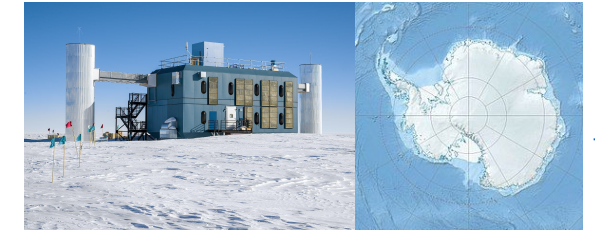
Embarrassingly parallel problems are the most natural fit for high-throughput computing systems.

Example HTC problems include:

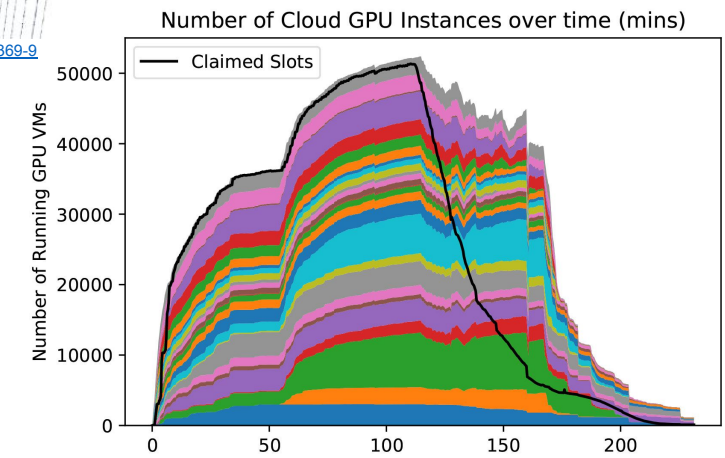
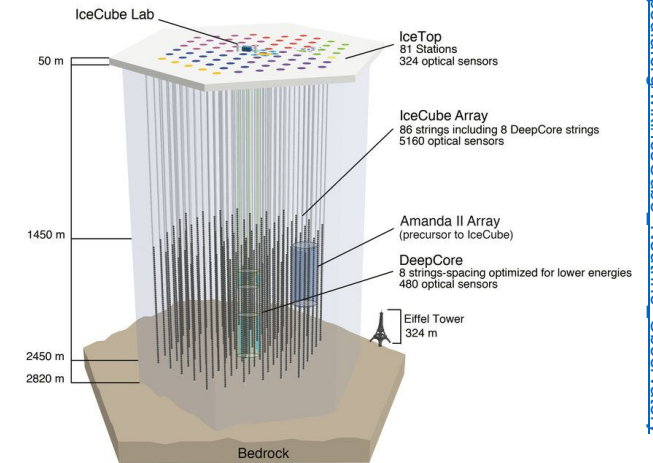
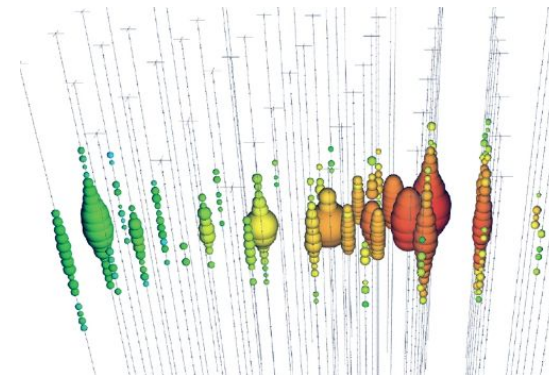
- [Batch processing](#)
- [Brute-force search](#)
- [Monte Carlo simulations](#)
- Parameter sweeps

Advantages of HTC vs. HPC applications:

- Simpler programming models - e.g., serial or shared-memory
- Leverage distributed and heterogeneous resources easily
- Resilient against job failures



https://en.wikipedia.org/wiki/IceCube_Neutrino_Observatory



[IceCube Neutrino Observatory, SDSC, WIPAC: GPU Cloud Burst Experiment](#)

Many-Task Computing (MTC)

Many-task computing (MTC) problems are comprised of an ordered set of *distinct* computational subtasks of *variable complexity*, often *coupled* via I/O operations.

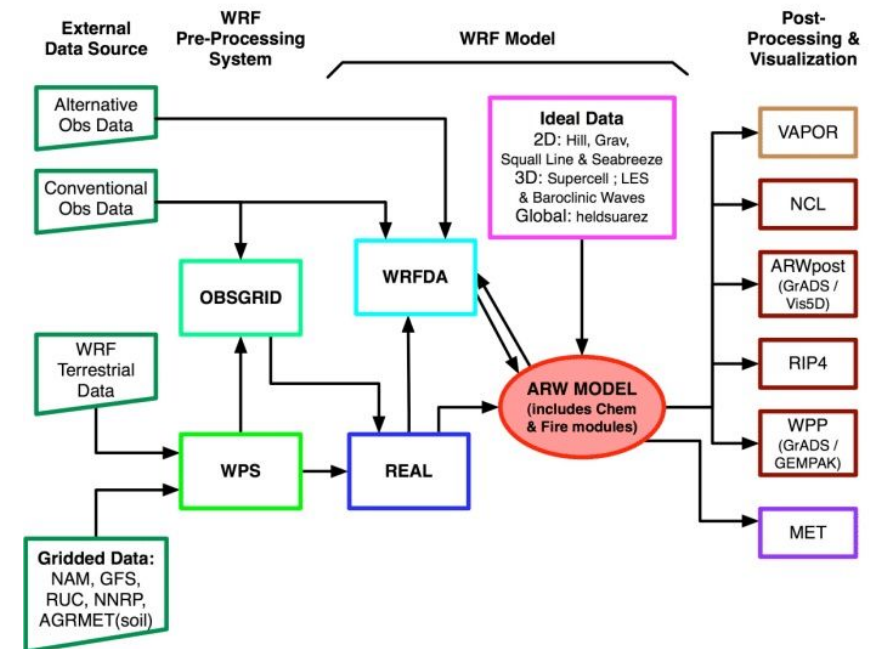
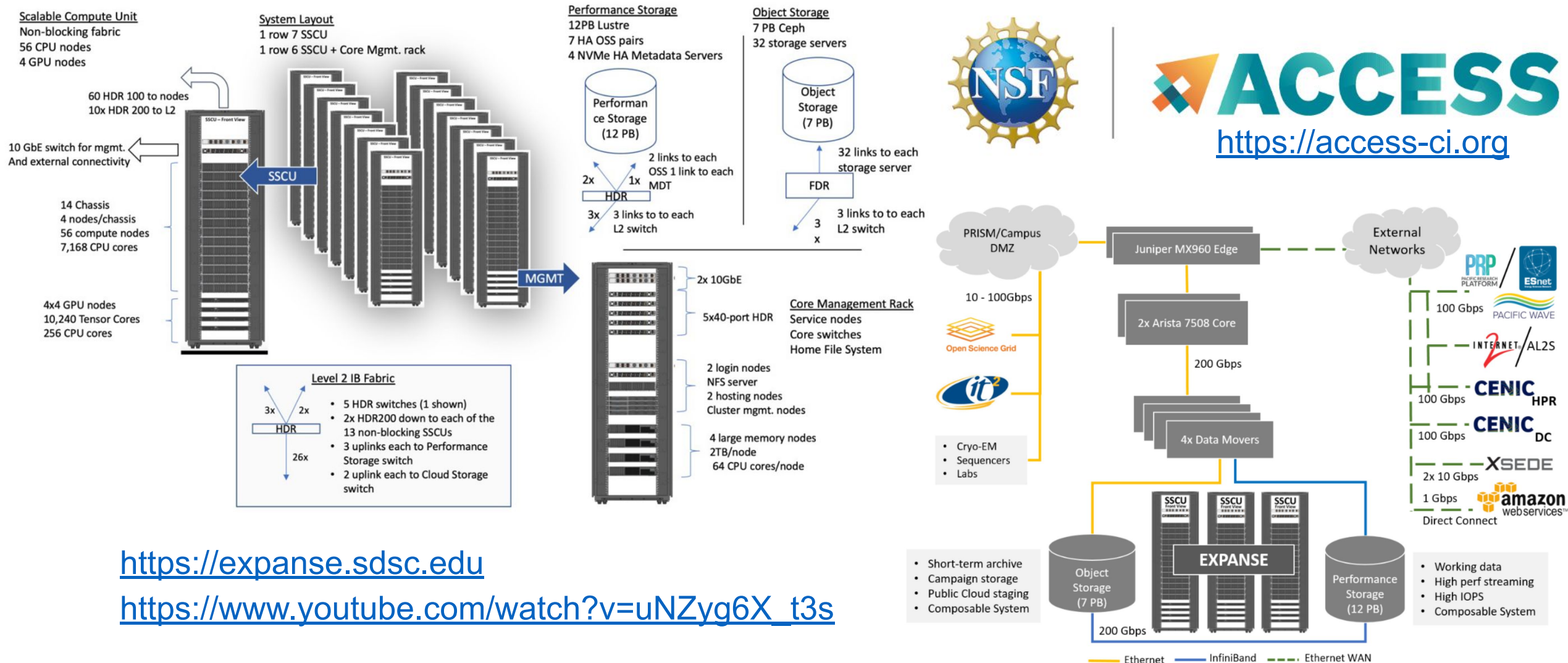


Table of Contents

- High-Throughput and Many-Task Computing
- **Batch Job Scheduling with Slurm**
- Job Arrays
- Job Dependencies
- Job Bundling
- Distributed High-Throughput Computing Resources
- Many-Task Computing and Workflow Managers
- Questions & Answers (30 min)

HPC System Architecture: Expanse @ SDSC



<https://expanse.sdsc.edu>

https://www.youtube.com/watch?v=uNZyg6X_t3s

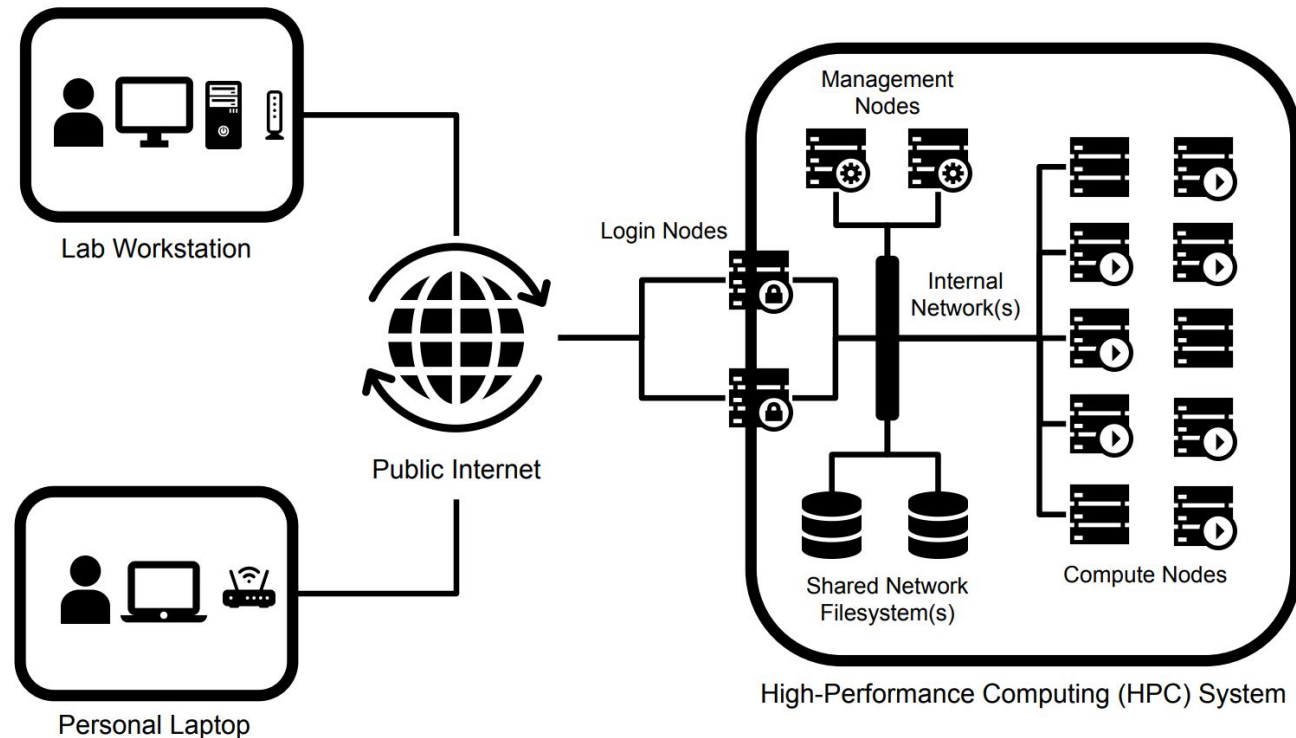
HPC System Architecture: Conceptual Model

Login node(s): Provide remote access to an HPC system; use only for simple tasks such as editing files, limited data transfers to and from the system, and batch job submission

Compute nodes: Run computational workloads: simulations, data analysis and visualization

Internal Network(s): Provide high-bandwidth, low-latency communication between compute nodes ; access to shared (parallel) filesystems; system management

Shared Network Filesystem(s): Provide input/output (I/O) access to data storage systems from any compute node



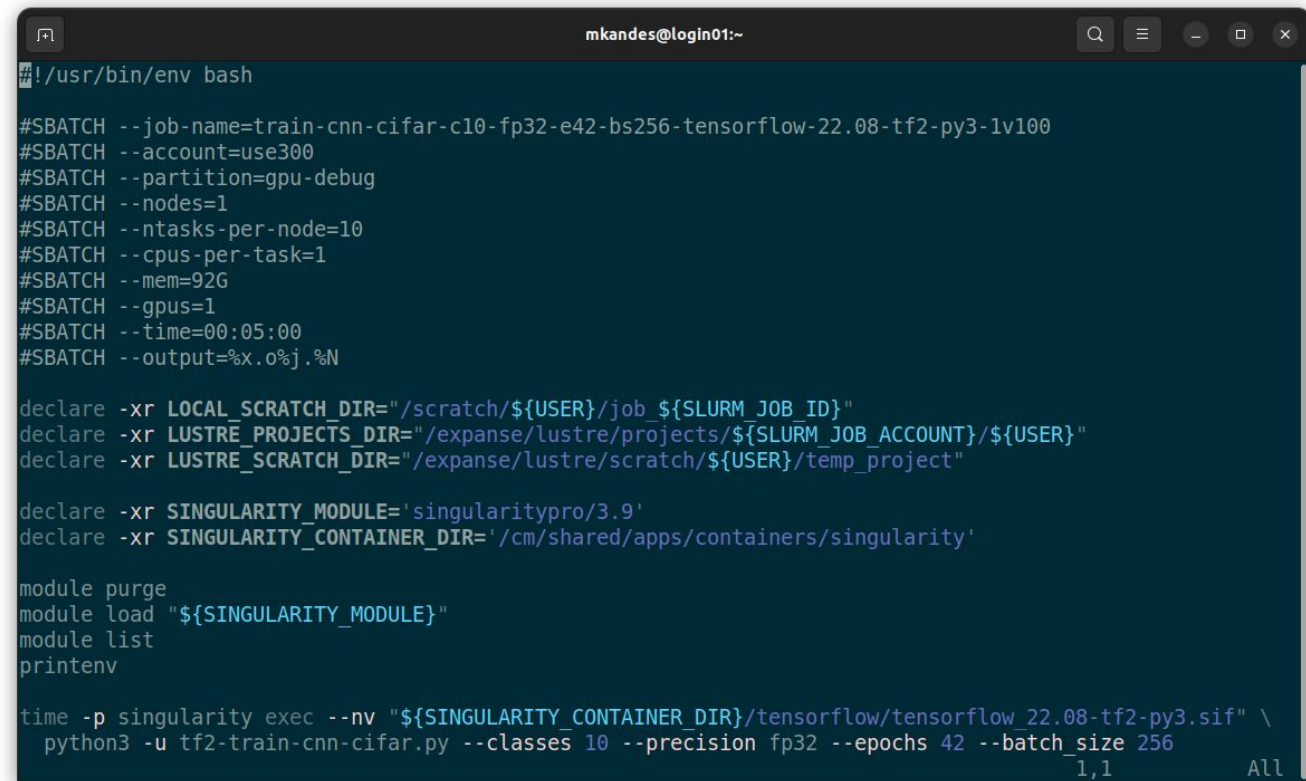
Management node(s): Run core system services such as cluster management software, system monitoring software, **batch job scheduler**, etc

What is a Batch Job?

A batch job is a pre-scripted sets of commands that are executed as processes, which may or may not be multi-threaded, on a certain type or set of dedicated compute resources within a computing system for a given amount of time without user interaction.

On HPC systems, batch jobs may run large-scale simulations, perform complex data analysis, or any other types of demanding computational tasks.

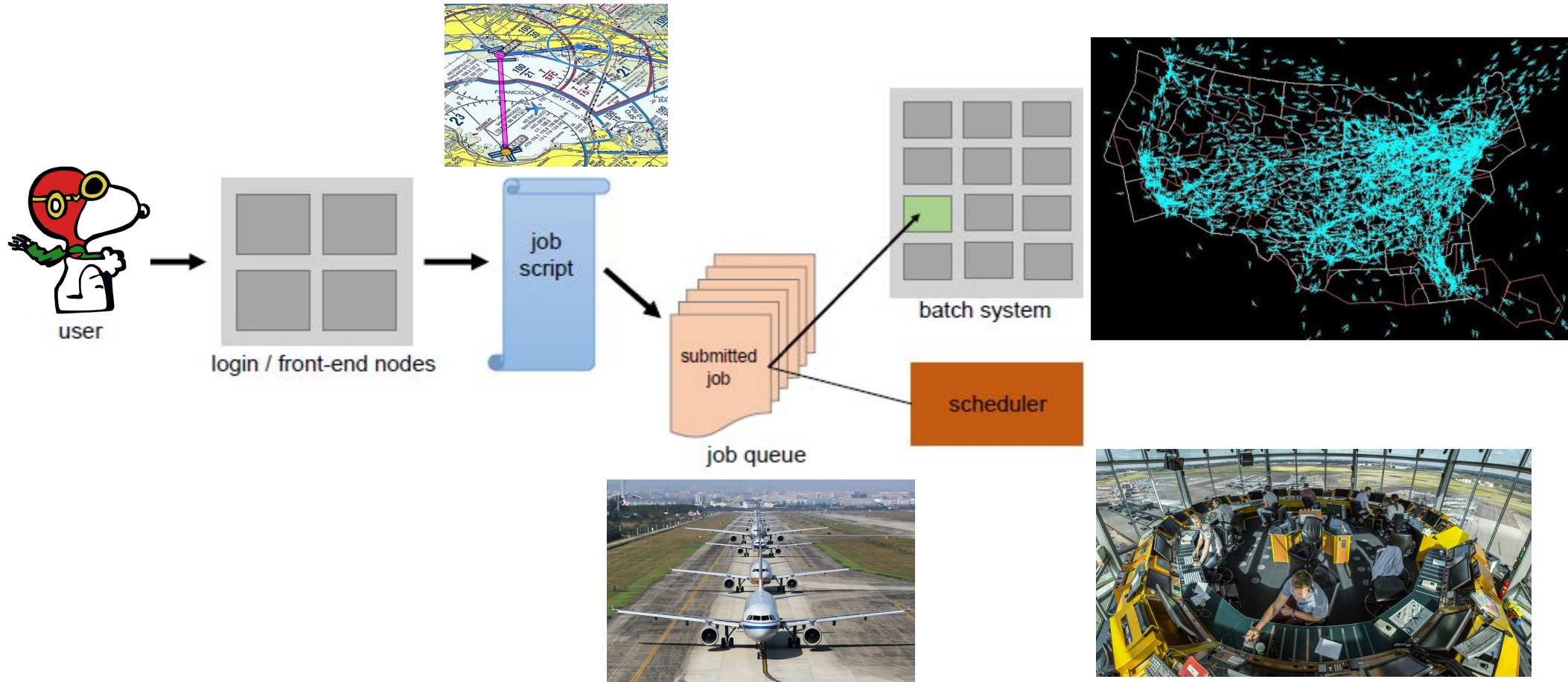
A batch job is typically specified as a shell script and then submitted to a queue of other batch jobs managed by a scheduler, which assigns the job to available resources and ensures that the job runs to completion.



```
mkandes@login01:~  
#!/usr/bin/env bash  
  
#SBATCH --job-name=train-cnn-cifar-c10-fp32-e42-bs256-tensorflow-22.08-tf2-py3-1v100  
#SBATCH --account=use300  
#SBATCH --partition=gpu-debug  
#SBATCH --nodes=1  
#SBATCH --ntasks-per-node=10  
#SBATCH --cpus-per-task=1  
#SBATCH --mem=92G  
#SBATCH --gpus=1  
#SBATCH --time=00:05:00  
#SBATCH --output=%x.0%j.%N  
  
declare -xr LOCAL_SCRATCH_DIR="/scratch/${USER}/job_${SLURM_JOB_ID}"  
declare -xr LUSTRE_PROJECTS_DIR="/exppanse/lustre/projects/${SLURM_JOB_ACCOUNT}/${USER}"  
declare -xr LUSTRE_SCRATCH_DIR="/exppanse/lustre/scratch/${USER}/temp_project"  
  
declare -xr SINGULARITY_MODULE='singularitypro/3.9'  
declare -xr SINGULARITY_CONTAINER_DIR='/cm/shared/apps/containers/singularity'  
  
module purge  
module load "${SINGULARITY_MODULE}"  
module list  
printenv  
  
time -p singularity exec --nv "${SINGULARITY_CONTAINER_DIR}/tensorflow/tensorflow_22.08-tf2-py3.sif" \  
python3 -u tf2-train-cnn-cifar.py --classes 10 --precision fp32 --epochs 42 --batch_size 256  
1,1 All
```


What is a Batch Job Scheduler?

A batch job scheduler is the software that controls and tracks *where* and *when* batch jobs submitted to a computing system will eventually run on its compute resources.



Slurm Workload Manager

The [Slurm Workload Manager](#) is a free and open-source distributed batch job scheduler and resource manager for Unix-like operating systems used by the majority of the world's supercomputers and computer clusters today.

Slurm provides three key functions: (1) the allocation of compute nodes for some period of time to process user jobs; (2) a framework for starting, executing, and monitoring the progress of user jobs across a set of allocated compute nodes; and (3) arbitrating the demand for compute nodes by managing a queue of pending user jobs.

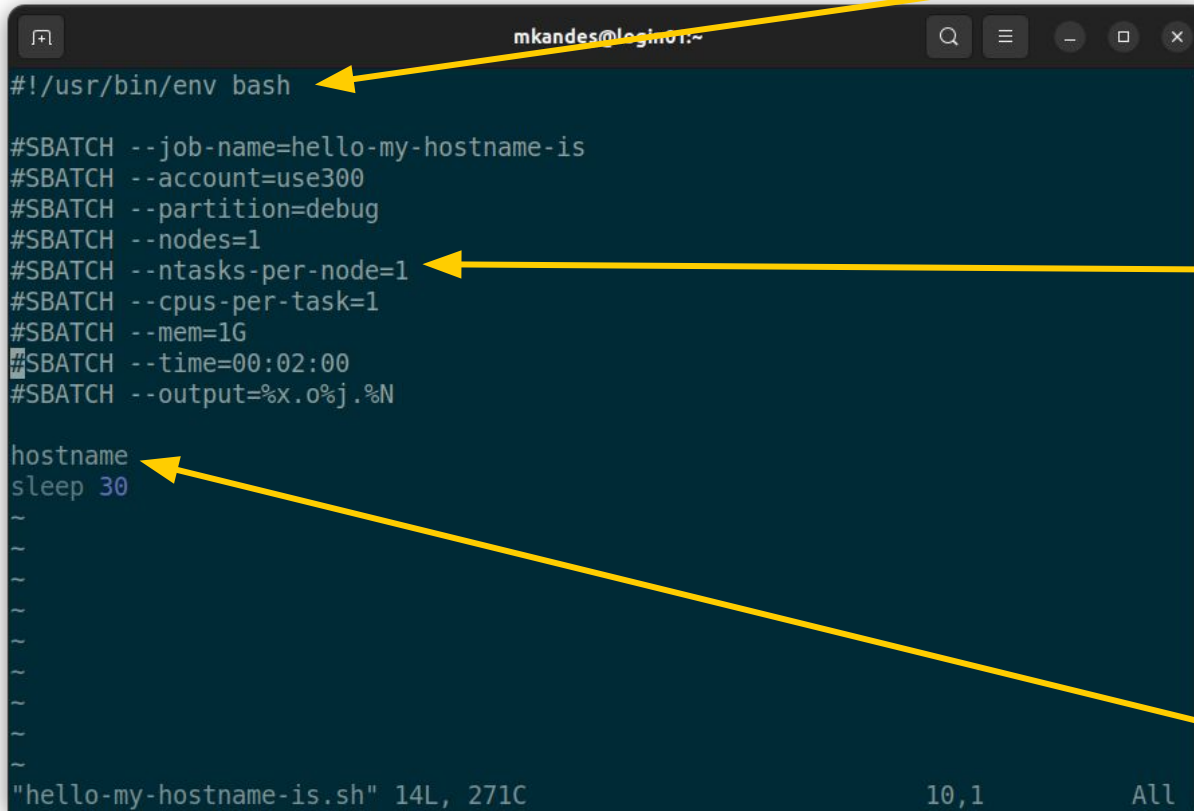
The essential Slurm user commands are:

- [sbatch](#) – submit a **batch job script** to the **scheduler**
- [squeue](#) – view information about **jobs** located in the **queue**
- [scancel](#) – cancel jobs, **job arrays** or job steps

For more information, see the Slurm documentation: <https://slurm.schedmd.com>



Exercise 1: Hello, my hostname is



```
mkandes@login01:~  
#!/usr/bin/env bash  
  
#SBATCH --job-name=hello-my-hostname-is  
#SBATCH --account=use300  
#SBATCH --partition=debug  
#SBATCH --nodes=1  
#SBATCH --ntasks-per-node=1  
#SBATCH --cpus-per-task=1  
#SBATCH --mem=1G  
#SBATCH --time=00:02:00  
#SBATCH --output=%x.o%j.%N  
  
hostname  
sleep 30  
  
"hello-my-hostname-is.sh" 14L, 271C 10,1 All
```

shebang (`#!`) character sequence is used to specify an interpreter that will execute the commands present in a plain text file.

#SBATCH directives are used to specify the computational resource requirements for your job and various other attributes of it. e.g., name of the job and output files, which account to charge the job to, etc.

All executable commands to be run must follow the list of `#SBATCH` directives specified in any job script.

Exercise materials are available at
<https://github.com/sdsc-complecs/batch-computing>

Table of Contents

- High-Throughput and Many-Task Computing
- Batch Job Scheduling with Slurm
- **Job Arrays**
- Job Dependencies
- Job Bundling
- Distributed High-Throughput Computing Resources
- Many-Task Computing and Workflow Managers
- Questions & Answers (30 min)

Job Arrays

Batch [job arrays](#) provide users with a simple and straightforward mechanism for submitting and managing *large collections* of *similar* jobs → ideal for HTC

Job arrays are supported by all major batch job schedulers, including [Slurm](#).

Typical characteristics of an array job:

- An array job is specified within a single batch job script submitted to the scheduler.
- Each job in an array is assigned an integer-valued index (in addition to a job ID).
- All jobs in an array have same resource request (e.g, cores, memory, time, etc).
- All jobs in an array run the same set of commands and only differ by the values of assigned to their common parameters and input values.

You can create an array job with Slurm by specifying the directive

#SBATCH --array=0-9

in your batch job script, where the argument can be specific array index values, a range of index values (shown above), and/or include an optional step size.



Exercise 2: Estimate Pi (with Slurm Job Arrays)

```
mkandes@login01:~  
#!/usr/bin/env bash  
#SBATCH --job-name=estimate-pi  
#SBATCH --account=use300  
#SBATCH --partition=shared  
#SBATCH --nodes=1  
#SBATCH --ntasks-per-node=1  
#SBATCH --cpus-per-task=1  
#SBATCH --mem=1G  
#SBATCH --time=00:30:00  
#SBATCH --output=%x.o%j.%N  
  
module purge  
time -p "${HOME}/4pi/bash/pi.sh" -b 8 -r 5 -s 10000  
  
"estimate-pi.sh" 15L, 311C written 14,0-1 All
```

Here, we'll estimate the value of Pi using a Monte Carlo method implemented in Bash.

The prepended time command measures the runtime of the estimation.

The input parameters to the **pi.sh** script are as follows:

- b** is the size of an integer in bytes;
- r** is number of digits that are retained after the decimal point in arithmetic operations; and
- s** is the number of samples to run.

Exercise materials are available at

<https://github.com/sdsc-complecs/batch-computing>

https://github.com/sdsc/sdsc-summer-institute-2025/tree/main/3.2_high_throughput_computing

Table of Contents

- High-Throughput and Many-Task Computing
- Batch Job Scheduling with Slurm
- Job Arrays
- **Job Dependencies**
- Job Bundling
- Distributed High-Throughput Computing Resources
- Many-Task Computing and Workflow Managers
- Questions & Answers (30 min)

Job Dependencies

Batch [job dependencies](#) allow users to defer the start of any job until a specified set of dependencies have been satisfied. These dependencies are typically based on the exit code of other jobs already submitted to the scheduler.

Job dependencies are supported by all major batch job schedulers, including [Slurm](#).

Enable complex workflows composed of **distinct** computational subtasks → ideal for MTC

You can impose a job dependency (or set of job dependencies) on submission for any Slurm job by appending the **--dependency (-d)** option to the **sbatch** command.

Slurm supports a number of job dependency types:

- **afterok:job_id[:jobid...]** - the submitted job can begin execution only after the specified jobs have successfully executed (i.e., ran to completion with an exit code of zero)
- ...



Table of Contents

- High-Throughput and Many-Task Computing
- Batch Job Scheduling with Slurm
- Job Arrays
- Job Dependencies
- **Job Bundling**
- Distributed High-Throughput Computing Resources
- Many-Task Computing and Workflow Managers
- Questions & Answers (30 min)

Job Bundling

Job bundling refers to a set of techniques used to manage a subset of computational tasks within a single batch job. They are most often used to implement HTC-like workflows on HPC systems that schedule their resources at the node-level.

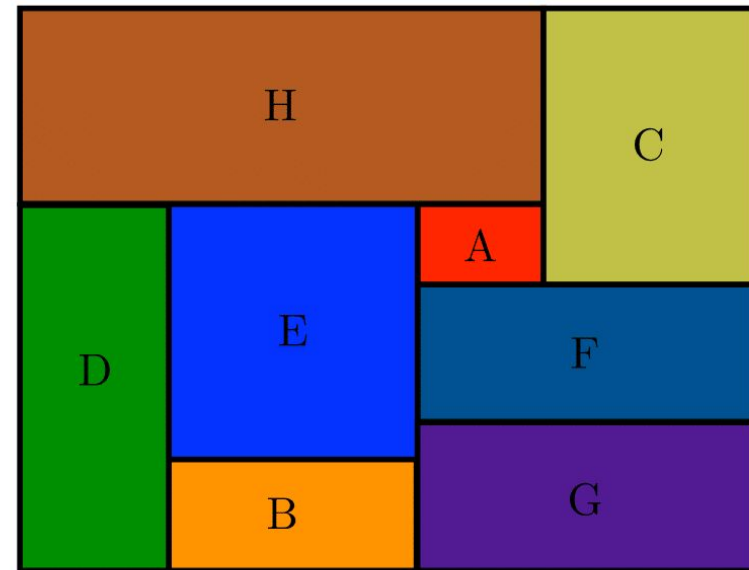
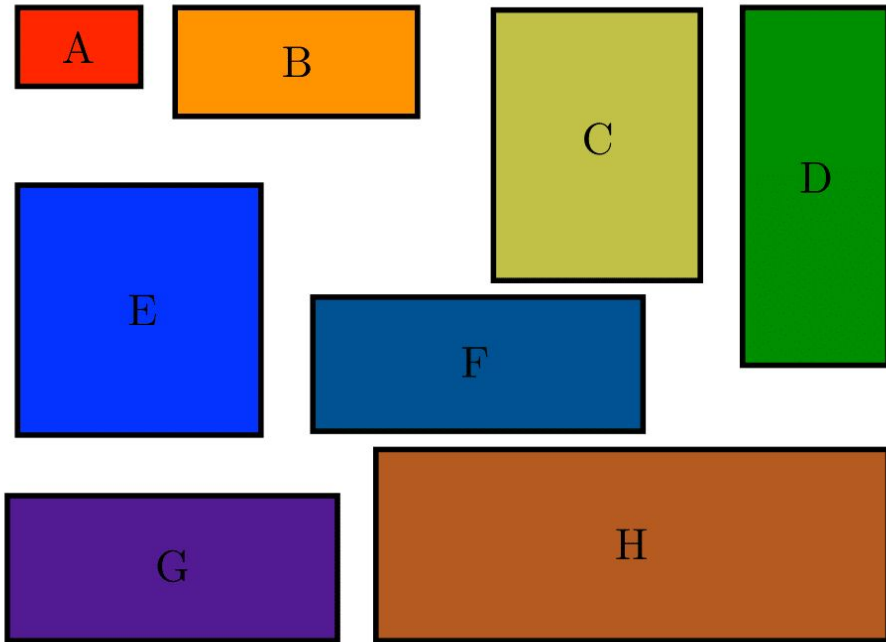


Table of Contents

- High-Throughput and Many-Task Computing
- Batch Job Scheduling with Slurm
- Job Arrays
- Job Dependencies
- Job Bundling Techniques
- **Distributed High-Throughput Computing Resources**
- Many-Task Computing and Workflow Managers
- Questions & Answers (30 min)

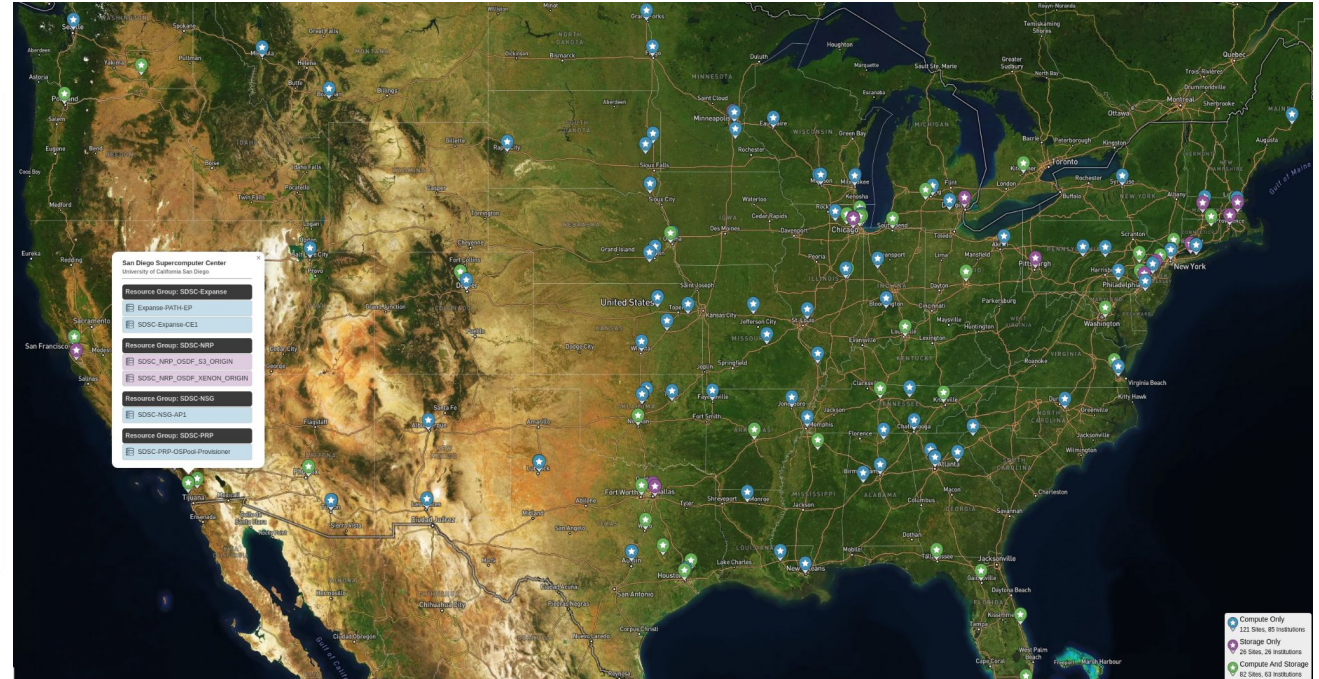
Open Science Grid (OSG)

The [Open Science Grid \(OSG\)](https://portal.osg-htc.org) is a distributed high-throughput computing (dHTC) system of shared compute, data storage, and network resources dedicated to the advancement of open science and technology.

	Ideal Jobs!	Still very advantageous	Maybe not, but get in touch!
Expected Throughput, per user	1000s concurrent cores	100s concurrent cores	Let's discuss!
CPU	1 per job	< 8 per job	> 8 per job
Walltime	< 10 hrs*	< 20 hrs*	> 20 hrs
RAM	< few GB	< 40 GB	> 40 GB
Input	< 500 MB	< 10 GB	> 10 GB**
Output	< 1 GB	< 10 GB	> 10 GB**
Software	pre-compiled binaries, containers	Most other than →	Licensed Software, non-Linux

*or checkpointable

** per job; you can work with a large dataset on OSG if it can be split into pieces



Sign up for your OSPool account at <https://portal.osg-htc.org>



Open Science Grid

Submit locally, run globally.



HTCCondor

Software Suite <https://htcondor.org>

PATh

The [Partnership to Advance Throughput Computing \(PATh\)](#) is funded by the National Science Foundation to advance the use of distributed High-Throughput Computing (dHTC) and its impact on research and education.

The PATh Facility is a purpose-built, national-scale dHTC resource dedicated to serving NSF-funded researchers.

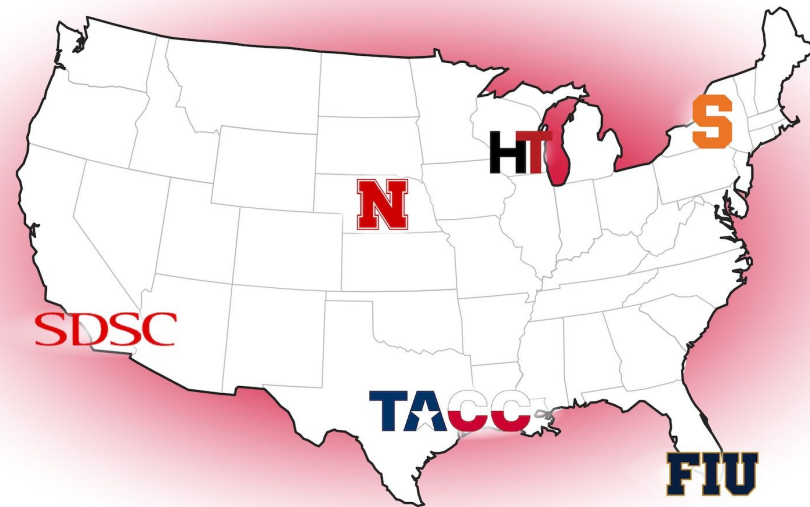
Apply for compute credits on PATh as part of a new NSF proposal or an existing award.

See NSF 22-051 DCL for more information:

<https://www.nsf.gov/pubs/2022/nsf22051/nsf22051.jsp>

The PATh Facility

Powering distributed high throughput computing



PATh PARTNERSHIP to ADVANCE
THROUGHPUT
COMPUTING

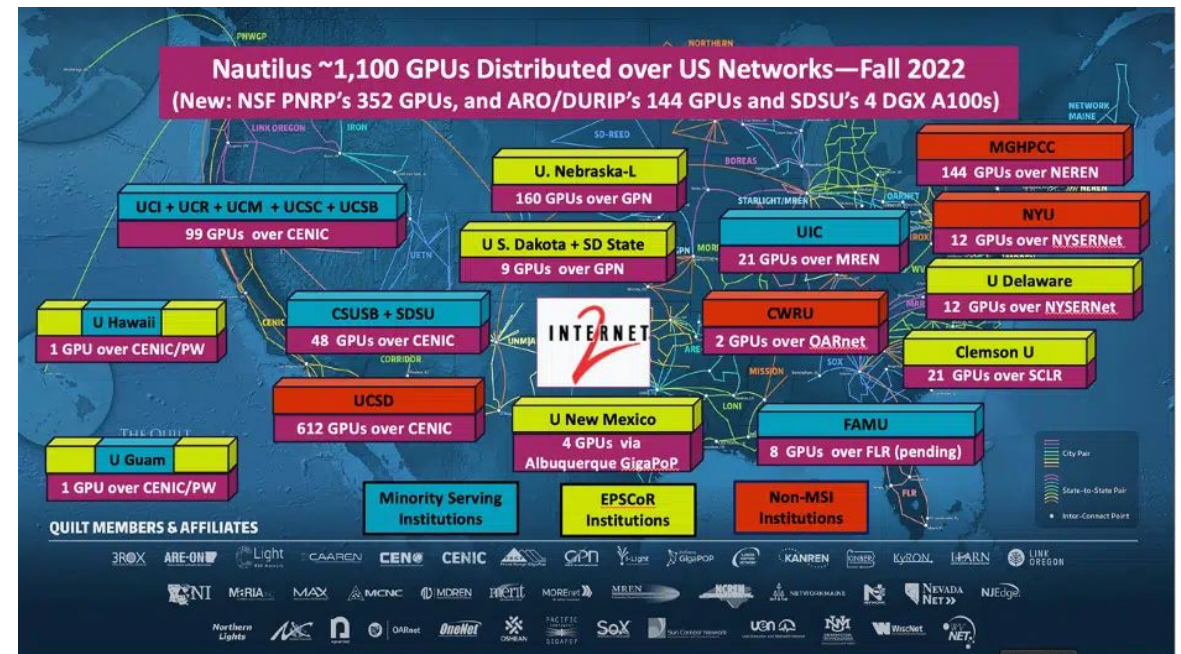
National Research Platform (NRP)

The [National Research Platform \(NRP\)](#) is a partnership of 50 institutions funded by the National Science Foundation led by researchers and cyberinfrastructure professionals that run a heterogeneous, distributed compute cluster named [Nautilus](#), which serves as a testbed for advanced compute, storage, and network resources.

https://www.sdsc.edu/support/user_guides/nrp.html



<https://kubernetes.io>



Get access to Nautilus at <https://docs.nationalresearchplatform.org/userdocs/start/get-access>

Table of Contents

- High-Throughput and Many-Task Computing
- Batch Job Scheduling with Slurm
- Job Arrays
- Job Dependencies
- Job Bundling Techniques
- Distributed High-Throughput Computing Resources
- **Many-Task Computing and Workflow Managers**
- Questions & Answers (30 min)

[Dask](#) is a Python library for parallel and distributed computing. Many of its computational features are built on its own [custom schedulers](#) that execute [task graphs](#).

Collections

(create task graphs)

Dask Array

Dask DataFrame

Dask Bag

Dask Delayed

Futures

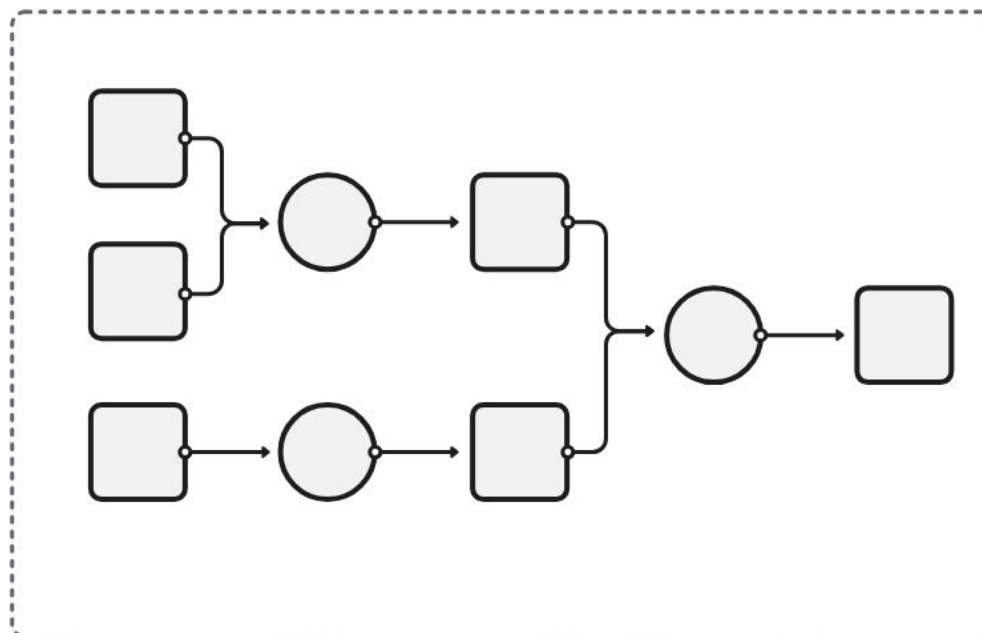


Task Graph



Schedulers

(execute task graphs)

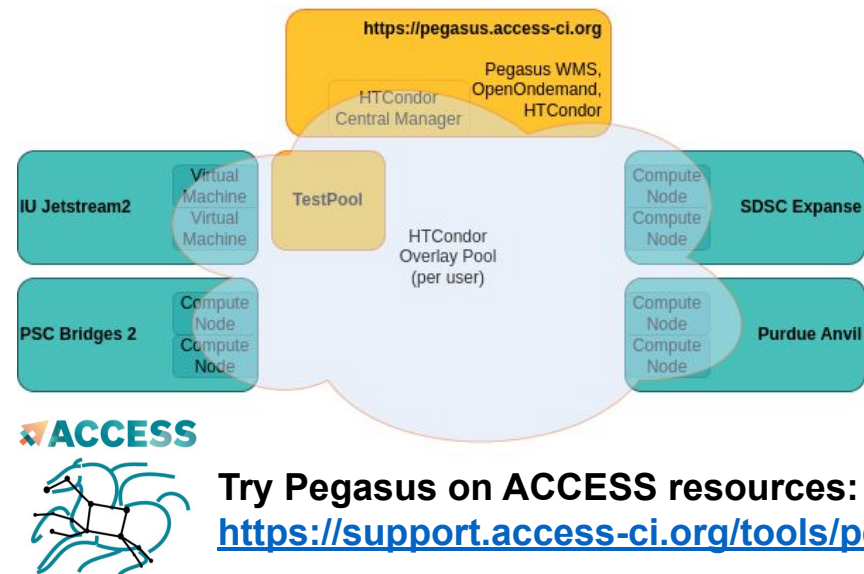


Single-machine
(threads, processes,
synchronous)

Distributed

Pegasus WMS

Pegasus is an open-source workflow management system that allows researchers to translate complex computational tasks into workflows that link and manage ensembles of dependent tasks and data files.



Try Pegasus on ACCESS resources:
<https://support.access-ci.org/tools/pegasus>

```
#!/usr/bin/env python3
import logging
from pathlib import Path

from Pegasus.api import *

logging.basicConfig(level=logging.DEBUG)

# --- Specify Input Files ---
input_data = File("data.csv")
rc = ReplicaCatalog().add_replica(
    site="local", lfn=input_data, pfn=Path(".").resolve() / "input_data/data.csv"
)

# --- Specify Executables ---
preprocess = Transformation(
    name="preprocess.py",
    site="local",
    pfn=Path(".").resolve() / "bin/preprocess.py",
    is_stageable=True,
)

findrange = Transformation(
    name="findrange.py",
    site="local",
    pfn=Path(".").resolve() / "bin/findrange.py",
    is_stageable=True,
)

analyze = Transformation(
    name="analyze.py",
    site="local",
    pfn=Path(".").resolve() / "bin/analyze.py",
    is_stageable=True,
)

tc = TransformationCatalog().add_transformations(preprocess, findrange, analyze)

# --- Build Workflow ---
wf = Workflow("analysis-workflow")

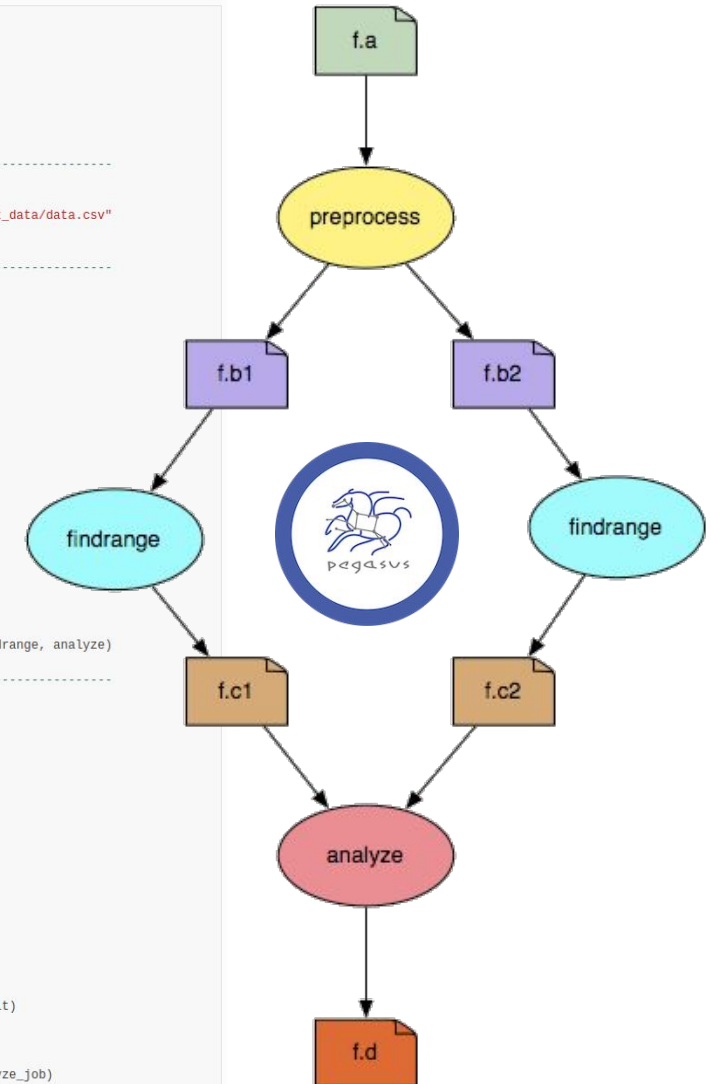
fb1 = File("f.b1")
fb2 = File("f.b2")
preprocess_job = (
    Job(preprocess)
    .add_args("arg1", "arg2")
    .add_inputs(input_data)
    .add_outputs(fb1, fb2)
)

fc1 = File("f.c1")
findrange_job1 = Job(findrange).add_inputs(fb1).add_outputs(fc1)

fc2 = File("f.c2")
findrange_job2 = Job(findrange).add_inputs(fb2).add_outputs(fc2)

result = File("result.csv")
analyze_job = Job(analyze).add_inputs(fc1, fc2).add_outputs(result)

wf.add_replica_catalog(rc)
wf.add_transformation_catalog(tc)
wf.add_jobs(preprocess_job, findrange_job1, findrange_job2, analyze_job)
```

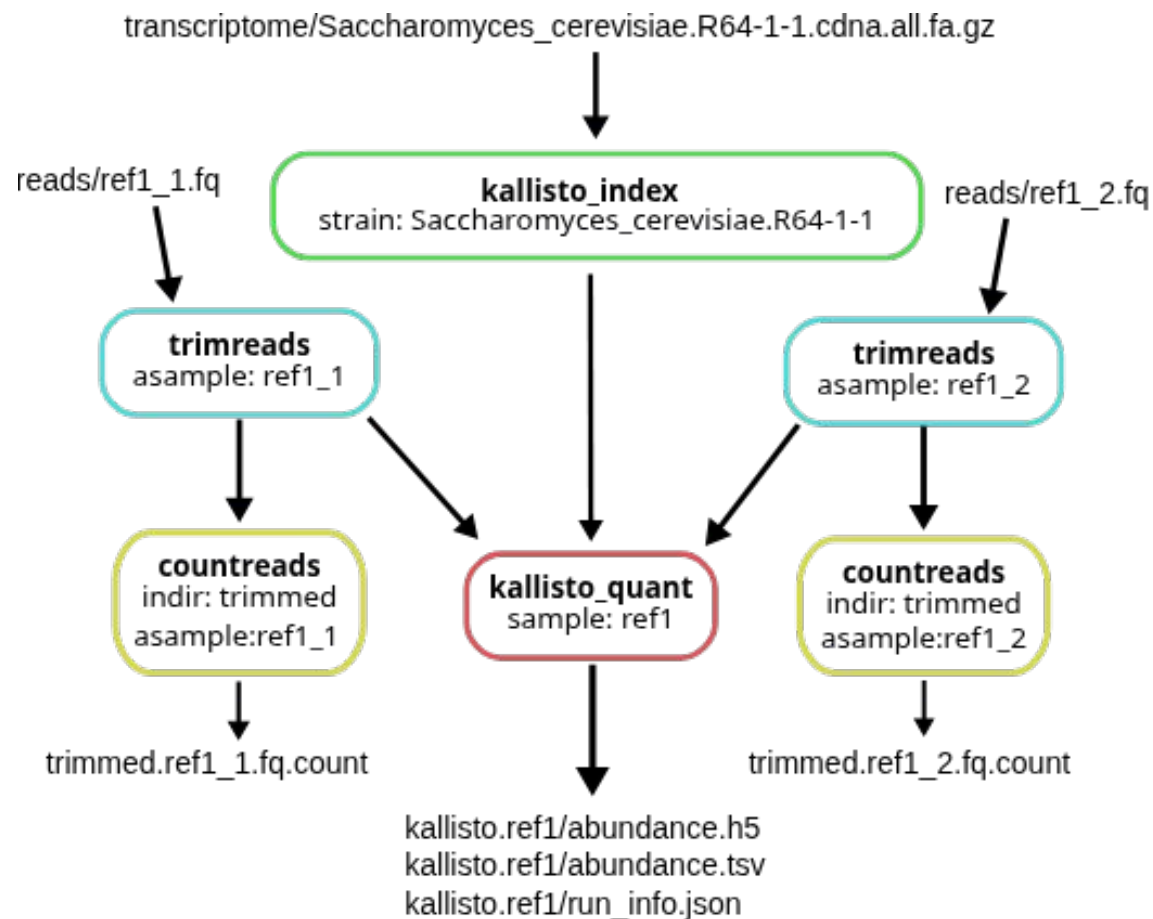


<https://www.youtube.com/watch?v=MNN80OHMQUQ>

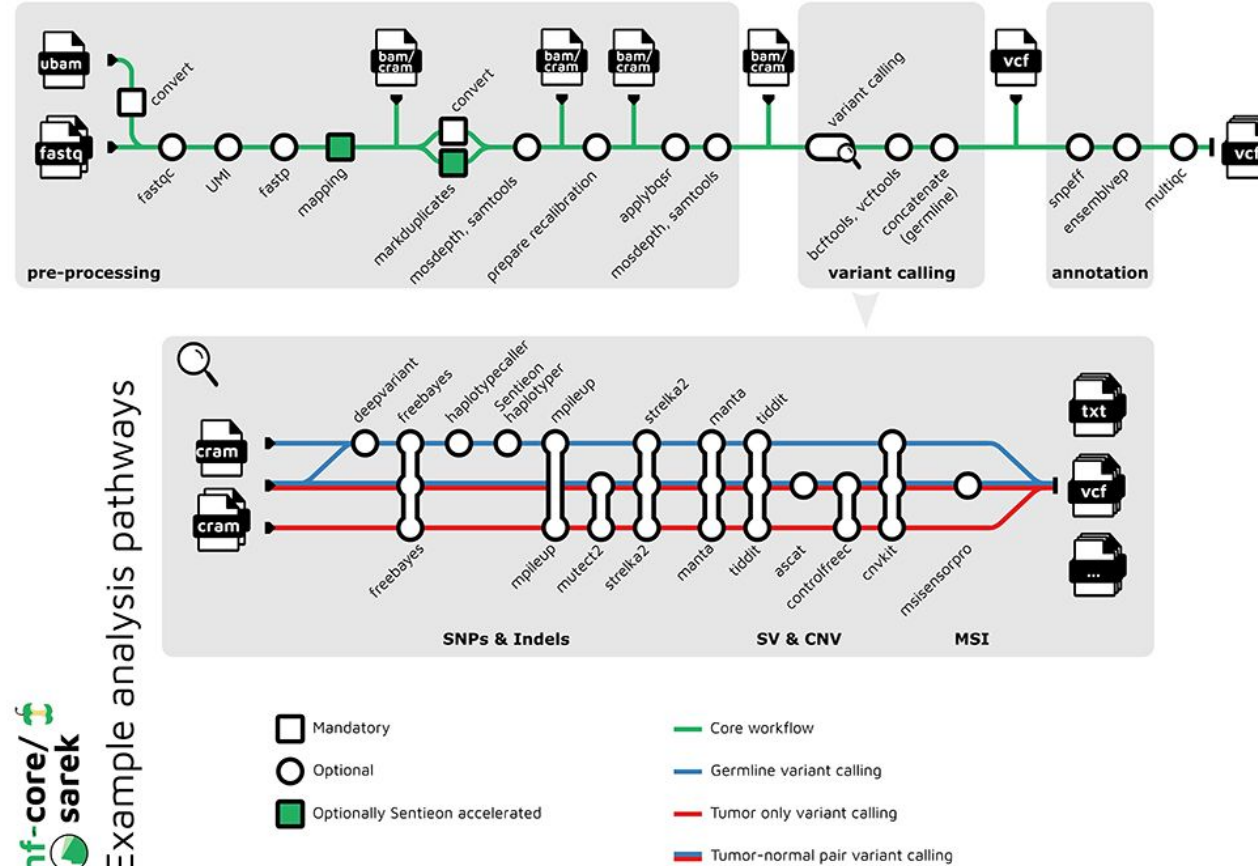
Snakemake



Snakemake is a workflow management system for creating reproducible and scalable data analysis pipelines.



[Nextflow](#) is a workflow management system predominantly used for bioinformatic data analysis.



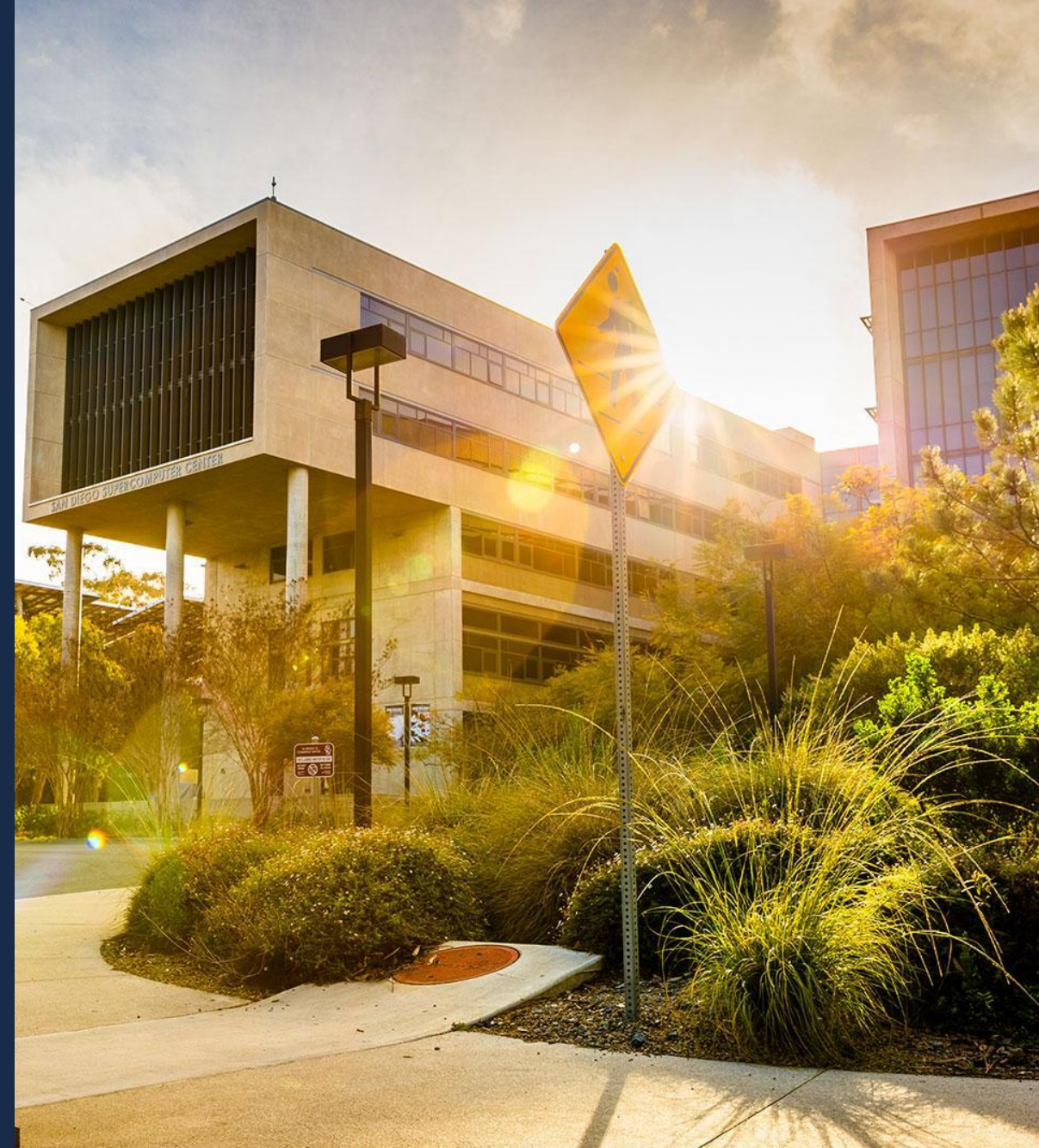
Adapted from: Fellows Yates, James A., et al. PeerJ 9 (2021).

Table of Contents

- High-Throughput and Many-Task Computing
- Batch Job Scheduling with Slurm
- Job Arrays
- Job Dependencies
- Job Bundling Techniques
- Distributed High-Throughput Computing Resources
- Many-Task Computing and Workflow Managers
- **Questions & Answers (30 min)**

Additional Information

- [Slurm Quick Start User Guide](#)
- [HPCWIKI: Scheduling Basics](#)
- [Introduction to HPC](#)
- [HPC in a Day](#)
- [Google's Shell Style Guide](#)



Questions & Answers

